

Contents

A small language model that reports its own embedding coordinates	1
Summary	1
1. The question	2
2. Terms	2
3. Setup	3
4. It is a working language model; the joint objective costs about 0.1 nats per token . . .	3
5. The answers are close to reading, not equal to it	4
5.1 Reading depends on the length of the embedding vector	4
5.2 The asked-about vectors keep ordinary lengths	5
6. The direction LayerNorm hides, and length versus direction	5
7. Editing an embedding vector inside the model (a consistency check)	6
8. What this does and does not show	6
9. Replication (seed 1)	7
10. Control: self-report with its input detached	7

A small language model that reports its own embedding coordinates

Summary

A small GPT-style model (4 layers, width 128, 1.35 million parameters) was trained on TinyStories to predict the next token and, at the same time, to answer “what is coordinate i of the embedding vector of token t ?” with a number. The embedding table is shared between input and output (tied), so the weights it reports on are the weights it uses to read and write text. Three such runs were trained, all for 15,000 steps, with a language-model-only baseline:

run	what differs	validation loss	centred R^2 , held-out tokens	follow, held-out tokens
language model only (seed 0)	no self-report	2.146		
joint, seed 0		2.248	0.680	0.751
joint, seed 1	another seed	2.290	0.504	0.666
control, seed 0	self-report cannot change the embedding table	2.290	0.982	0.868

(Held-out tokens are never asked about in training. Follow: how far the answer about coordinate i moves when coordinate i of the embedding vector is nudged; 1 for exact reading. Centred R^2 : 1 for exact answers, 0 for answering each coordinate’s average. Section 2 defines all terms.)

- All three are working language models (coherent short stories). Self-report costs 0.10 nats per token (joint, seed 0) or 0.14 (control) against the baseline.
- When the self-report cannot change the embedding table (control), it reads the embedding well and generalizes fully: the same accuracy for tokens never asked about as for tokens asked about (centred R^2 0.982 and 0.982; follow 0.87 and 0.87).
- When it can (joint runs), it reshapes the embedding vectors of the tokens it is asked about, and the reading generalizes worse to the other tokens (centred R^2 0.68 and 0.50). The clearest case: language modelling makes the embedding vectors of rare tokens long (about 4); in the joint runs, the rare tokens that are asked about stay at ordinary length (about 1.5), the rare tokens that are not stay long, and the reader, fitted to the ordinary ones, fails on

them. In the control, all rare vectors stay long, the reader sees long ones in training, and reads them well.

- In every run the answers follow changes of an embedding vector's direction better than changes of its length; this is tied to a direction that LayerNorm hides from the first block (section 6). An exact consequence of LayerNorm (a change along the all-ones direction moves every answer by the same amount) holds to rounding error.
- Editing a token's embedding vector inside the model moves the self-report about that token 0.72 to 0.83 of the way on average, depending on the run (frequent tokens), and changes the language model's predictions mainly right after that token.

1. The question

Can a language model answer questions about its own weights, and do the answers come from the weights themselves? The test is the one used in `../self-report-embeddings/`: change a weight and see whether the answer changes with it. There, a toy model with random embeddings learned to read its embedding nearly exactly once it was trained on thousands of distinct embedding vectors. Here the embeddings are shaped by language modelling and shared with the output layer, and the model has the standard parts of a language model (positional embeddings, LayerNorm).

2. Terms

- Token: one of the 4,096 symbols of the tokenizer (byte-level BPE trained on TinyStories), including an end-of-story token.
- Embedding table: vectors in $\mathbb{R}^{128}$, one for each token and one for each of 129 extra symbols (QUERY, COORD_0, ..., COORD_127). Used at the input (the first internal state at a position is the embedding vector of the symbol there plus a positional vector $P[\text{position}]$) and, for the 4,096 tokens, at the output (tied: the score of a possible next token is the inner product of the final, normalized internal state with that token's embedding vector).
- Embedding vector of token t : $E[t] \in \mathbb{R}^{128}$; coordinate i is $E[t, i]$. Length: the Euclidean norm $|E[t]|$.
- Block: one of the 4 transformer layers (attention, then feed-forward). Each block applies LayerNorm to its input before using it: subtract the mean of the 128 coordinates, divide by their standard deviation, then apply a learned scale and shift per coordinate.
- Question (t, i) : the three-symbol sequence [QUERY, COORD_ i , t].
- Answer: the number the model outputs for a question: $w \cdot h + b$, where h is the internal state at the last position (t 's) after the last block and before the final LayerNorm, and $w \in \mathbb{R}^{128}$, b are the number head's weights.
- Correct answer: $E[t, i]$, the current value.
- Training tokens: 3,072 of the 4,096 tokens (three quarters, chosen at random by the seed), about which the model is asked during training. Held-out tokens: the other 1,024, never asked about. 88 training tokens and 47 held-out tokens never occur in the training text; all others do.
- Validation loss: the average over 100×32 fixed windows of 128 tokens from the TinyStories validation set of $-\log(\text{probability given to the actual next token})$, in nats per token. (The training logs use a smaller set of 20×32 windows; section 4 says when.)
- R^2 : $1 - \Sigma(\text{answer} - \text{correct answer})^2 / \Sigma(\text{correct answer} - \text{mean})^2$, with one mean over all questions in the set.
- Centred R^2 : the same, with the mean of each coordinate over the tokens of the set in place of the single mean, so that answering each coordinate's mean scores 0. Needed because the embedding vectors share a common component: answering the coordinate means alone gives R^2 0.58 on training tokens and 0.49 on held-out tokens.
- Random vectors: 1,024 vectors drawn from a normal distribution with the mean and spread of each coordinate of the 4,096 embedding vectors. Put at t 's position in place of an embedding vector; question i asks for coordinate i of the random vector.

- J: for token t , the 128×128 matrix of derivatives of the 128 answers about t with respect to the 128 coordinates of $E[t]$.
- Reading: $J = I$ (every answer moves exactly with its coordinate, and with nothing else).
- Follow: $\text{tr}(J)/128$, the average over coordinates i of how much the answer about coordinate i moves per unit change of coordinate i ; reported as the average over tokens. 1 for reading; 0 for answers that do not change when $E[t]$ changes slightly. (With the other weights fixed, every answer is a function of $E[t]$; follow measures how close that function is to the identity near $E[t]$.)
- Other movement: for each token, $\|J - f \cdot I\|_F / \sqrt{128}$ with f that token's follow, averaged over tokens: the root-mean-square size, over random directions of a small change δ , of the movement of the answers that is not $f \cdot \delta$, as a fraction of $|\delta|$. 0 for reading.
- Finite-change follow: add to $E[t]$ a vector δ in a random direction with length 10% of $|E[t]|$; the 128 answers about t change by Δ ; the value is $\Delta \cdot \delta / |\delta|^2$.

3. Setup

- Data: TinyStories training shard 0 of 4 (529,930 stories, 122 million tokens) and the validation set (21,990 stories, 4.9 million tokens).
- Model: GPT-style decoder, pre-LayerNorm, learned positional embeddings, 4 blocks, width 128, 4 attention heads, feed-forward width 512 (GELU), context 128 tokens. 1,350,657 parameters, of which 540,800 in the embedding table.
- Training, all runs: 15,000 steps of AdamW (learning rate $2 \cdot 10^{-3}$ after 200 warm-up steps, cosine decay to $2 \cdot 10^{-4}$; weight decay 0.1 on the block and number-head matrices only; gradient norm clipped at 1, for the combined gradient). Each step uses 32 windows of 128 tokens of text (61 million tokens seen in all, half the training shard).
- Joint run: each step also asks 256 random questions about training tokens; loss = language-modelling loss + $(1 - R^2 \text{ of those 256 answers})$. No gradient flows through the correct answers, so the self-report changes the embedding table only through its use as the input at t 's position.
- Language-model-only run (baseline): the same without the questions, and the same seed (0): same initial weights, same text windows in the same order.
- Each run took about 2 to 2.3 hours on 2 CPU threads.
- Checks: a separate Claude instance reviewed the code before the runs, and another reviewed a draft of this report; their findings were checked and fixed (NOTES.md). 15 tests, including checks of every measure on hand-made functions with known answers, and an exact test of the architectural fact in section 6.

4. It is a working language model; the joint objective costs about 0.1 nats per token

Final validation loss (100×32 windows): joint 2.248, language model only 2.146, difference 0.102.

During training (training logs, 20×32 windows):

step	joint	language model only	difference
1,500	3.094	2.854	0.240
4,500	2.609	2.487	0.122
9,000	2.388	2.284	0.104
15,000	2.261	2.154	0.107

Observation. The joint model is worse throughout. The difference shrinks during the first half of training and stays between 0.10 and 0.11 from step 7,500 on. The language-model-only run reached the joint run's final loss at about step 9,700.

This is the cost of the joint objective under this optimizer (shared Adam state, one gradient clip for the combined gradient), not necessarily a cost in model capacity; the two are not separated here.

Three stories sampled from the joint model (prompt “Once upon a time”, temperature 0.8) are in results/measure_joint_s0.json. One of them:

Once upon a time, there was a little girl named Lily. She loved to play with her toys and her toys. One day, she went to the park with her mommy. She saw the big red ball that was round and shiny. Her mommy said, “Let’s take it home.” Lily was sad and started to cry. ...

Another is less coherent (“The knight was never afraid of the knight’s heart, even though the pretending him special adventure is there to be something to be”).

5. The answers are close to reading, not equal to it

Measured on the joint model after training. Jacobian measures use 512 random training tokens and 512 random held-out tokens.

measure	training tokens	held-out tokens
R ²	0.990	0.838
centred R ²	0.976	0.680
follow (spread over tokens)	0.851 (0.02)	0.751 (0.16)
other movement	0.41	0.40
finite-change follow, mean (median)	0.848 (0.851)	0.748 (0.798)

Random vectors: R² 0.914, centred R² 0.813.

Baseline: the language-model-only model’s number head was never trained, so it gives R² −2.55 and −2.03 and follow −0.001 on the same sets.

Observation. The answers depend on the current embedding vector in the right direction: a small change of coordinate *i* moves the answer about it 0.85 (asked about) or 0.75 (never asked about) of the way, and real changes of 10% give the same mean. The answers also move by about 0.4 of the change in other ways. This is far from the toy model at 4,096 tokens (follow 0.997 to 1.005, other movement 0.06 to 0.10).

For comparison, not like for like: a linear map from the internal state of the language-model-only model at *t*’s position (question layout, before the final LayerNorm) to E[*t*], fitted by ridge regression on the training tokens, gives centred R² 0.834 on the held-out tokens. That map reads all 128 coordinates from one state; the model’s number head must answer one coordinate per question through one shared readout.

5.1 Reading depends on the length of the embedding vector

(This and section 5.2 describe the joint run; section 10 shows how the control differs.)

On held-out tokens, follow depends on how often a token occurs in the training text (counts in the 122 million training tokens; 64 tokens per bin):

tokens	count	number of tokens	follow	mean length of E[<i>t</i>]
training	below 100	248	0.85	1.53
training	100 to 9,999	1,935	0.85	1.44
training	10,000 or more	889	0.85	1.33
held-out	below 100	97	0.34	3.95

tokens	count	number of tokens	follow	mean length of E[t]
held-out	100 to 9,999	629	0.79	1.75
held-out	10,000 or more	298	0.83	1.41

The 97 rare held-out vectors point in nearly the same direction (mean pairwise cosine 0.998), so they are close to a single test point.

Rescaling test. Score: $1 - (\text{mean squared error per vector}) / (\text{mean squared distance of the held-out vectors from their coordinate means})$.

held-out vectors	as they are	rescaled, direction kept
rare (mean length 3.95)	-1.34	0.96 at length 1.41
frequent (mean length 1.41)	0.94	-1.67 at length 3.95

Observation. Length alone decides whether the reading works: the same directions are read well at the length of the vectors the reader was trained on and badly at 2.8 times that length.

5.2 The asked-about vectors keep ordinary lengths

Mean length of E[t] by frequency:

model	tokens	below 100	100 to 9,999	10,000 or more
language model only	training	4.68	1.65	1.44
language model only	held-out	4.77	1.68	1.46
joint	training	1.53	1.44	1.33
joint	held-out	3.95	1.75	1.41

Observation. Language modelling alone makes rare tokens' vectors long (4.7, about three times the length of the other tokens' vectors). In the joint model, rare tokens that are asked about have ordinary lengths (1.53); rare tokens that are not asked about stay long (3.95). Their directions changed too: the mean pairwise cosine of the rare asked-about vectors is 0.91, against 0.99 in the language-model-only model.

Two explanations fit this run: 1. The self-report's gradient through its input pulls the asked-about vectors into the range where its reading works. 2. An optimizer effect: with Adam and no weight decay on the embedding table, a small but consistent gradient on a rarely used row produces full-size steps, which would make rare vectors drift long; any added noisy gradient on those rows (here, the self-report's) enlarges Adam's normalization for them and slows that drift, whatever its direction.

The control run (section 10) shows that the self-report's gradient on the embedding table is what keeps the asked-about vectors short: without it, they grow long like the others. It does not separate the two explanations, since both act through that gradient.

6. The direction LayerNorm hides, and length versus direction

The first block sees the internal state at t's position, $E[t] + P[2]$, only through LayerNorm, which subtracts the mean of the coordinates and divides by their standard deviation. Two directions are therefore special:

- The all-ones direction (1, ..., 1). A change of E[t] along it changes only the mean, which every block removes, so the blocks never see it; it reaches the answers only through the number

head's direct readout of the internal state, and moves every answer by exactly Σw . Measured: J·1 has all entries equal to $\Sigma w = 0.7466$; the spread over the 128 answers averages $1.2 \cdot 10^{-7}$ over 512 tokens (single-precision rounding). This holds for any weights, and is tested.

- The direction c of the centred vector $E[t] + P[2] - \text{mean} \cdot (1, \dots, 1)$. Scaling c changes nothing that the first block sees. Later blocks could see the change through the internal state, but the trained model barely does: the response of the answers along $u = c/|c|$ ($u^T J u$) is 0.005 for training and held-out tokens (64 tokens each).

The length response follows from the second point. A change along $E[t]$ (lengthening it) is partly a change along c : the cosine between $E[t]$ and c is 0.62 (training tokens) and 0.68 (held-out tokens), because $P[2]$ has length 1.36, comparable to $E[t]$. Measured responses along $E[t]$: 0.65 and 0.50, against 0.85 and 0.75 on average over the directions perpendicular to $E[t]$. So answers follow changes of direction better than changes of length, consistent with the length direction overlapping the direction the model cannot see. This may also be part of why the rare held-out vectors of section 5.1 are read badly: they differ from the vectors the reader learned mainly in length, and length lies partly along that direction. I have not tested this further.

7. Editing an embedding vector inside the model (a consistency check)

For each of the 20 most frequent held-out tokens in the validation text (among them "it", "he", "in", "said", "she", "his", "day"), I added to its embedding vector, inside the model, a change of 10% of its length in a random direction, and measured the answers about that token and the next-token predictions on 20×32 validation windows.

- Self-report: finite-change follow 0.73 to 0.90 (mean 0.82). These are frequent tokens; for rare held-out tokens follow is about 0.34 (section 5.1).
- Language model: the Kullback-Leibler divergence of the edited from the original next-token distribution was larger at positions right after the edited token than at positions with no earlier occurrence of it, for 19 of the 20 tokens. Averaged over the 20 tokens: $5.0 \cdot 10^{-3}$ against $1.6 \cdot 10^{-4}$ nats (32 times larger); pooled over positions, 27 times.

Both changes are guaranteed in kind, since the edited vector is the input at the token's position and is used at the output: this section checks that the self-report and the language model both respond, and how much; it does not test a relation between the two responses.

8. What this does and does not show

This section takes the control (section 10) into account.

- A small working language model can learn to answer questions about its own embedding coordinates with answers that follow the current embedding: in the control, follow 0.87 and centred R^2 0.98 on tokens never asked about, at a cost of 0.14 nats per token in language modelling under this optimizer.
- If the self-report is allowed to change the embedding table, it changes the vectors it is asked about, and its reading then generalizes worse to the other tokens (follow 0.75 and 0.67, centred R^2 0.68 and 0.50, in two seeds); the language model pays less (0.10 nats per token, seed 0). The poor generalization is a shift between the asked-about and the other embedding vectors, created by the self-report itself.
- Reading is not exact in any run: answers also move in other ways by 0.35 to 0.48 of a change, and changes of length are followed less than changes of direction. Whether pre-LayerNorm is why reading is less exact than in the toy model (follow 0.997 to 1.005) is a hypothesis: the toy model also had random embeddings, no language modelling, and a different budget.
- As in the toy model, the embedding vector is the model's input at t 's position, so this is reading an input. Weights that are not inputs remain the open case (PLAN.md, roadmap).
- Not tested: a reader trained on a frozen language model; other loss weights; other sizes; a seed-1 control. One baseline seed.

9. Replication (seed 1)

A second joint run with seed 1: different initial weights, different text order, and a different random choice of the 1,024 held-out tokens. There is no language-model-only run with seed 1, so its language-modelling cost cannot be measured directly.

measure	seed 0	seed 1
validation loss	2.248	2.290
centred R^2 , training tokens	0.976	0.955
centred R^2 , held-out tokens	0.680	0.504
centred R^2 , random vectors	0.813	0.699
follow, training tokens	0.851	0.745
follow, held-out tokens	0.751	0.666
other movement, training / held-out	0.41 / 0.40	0.48 / 0.46
response along $E[t]$ (length), training / held-out	0.66 / 0.49	0.57 / 0.42
response along the hidden direction c, training / held-out	0.005 / 0.006	0.022 / 0.015
follow by frequency, held-out: below 100 / 100-9,999 / 10,000+	0.34 / 0.79 / 0.83	0.27 / 0.68 / 0.73
mean length, rare held-out / rare training	3.95 / 1.53	4.33 / 1.66
rescaling test, rare held-out: as is $\rightarrow$ shrunk	-1.34 $\rightarrow$ 0.96	-3.47 $\rightarrow$ 0.90
rescaling test, frequent held-out: as is $\rightarrow$ grown	0.94 $\rightarrow$ -1.67	0.91 $\rightarrow$ -2.75
edit test: self-report finite-change follow, range (mean)	0.73 to 0.90 (0.82)	0.61 to 0.90 (0.72)
edit test: KL right after the token / with no earlier occurrence	32 times (19 of 20 tokens)	34 times (20 of 20 tokens)

Observation. Every qualitative result of seed 0 holds for seed 1: partial reading, weaker for held-out tokens; near-blindness along the hidden direction; weaker response to length than to direction; failure on long rare held-out vectors that disappears when they are rescaled; ordinary lengths for rare asked-about vectors. The size of the effects varies a lot between the two seeds: centred R^2 on held-out tokens is 0.68 against 0.50, and follow 0.75 against 0.67. With two seeds, the numbers in sections 4 to 7 should be read as one sample, not as estimates with known uncertainty.

10. Control: self-report with its input detached

Same as the joint seed-0 run (same initial weights, same text, same questions in the same order), except that the self-report sees $E[t]$ with its gradient blocked: it trains the reader (the blocks, the number head, the extra symbols' embedding vectors) but cannot change the text tokens' embedding vectors, which are then shaped by language modelling alone.

measure	joint, seed 0	control, seed 0
validation loss (baseline 2.146)	2.248	2.290

measure	joint, seed 0	control, seed 0
centred R^2 , training tokens	0.976	0.982
centred R^2 , held-out tokens	0.680	0.982
centred R^2 , random vectors	0.813	0.821
follow, training / held-out tokens	0.851 / 0.751	0.871 / 0.868
other movement, training / held-out	0.41 / 0.40	0.35 / 0.35
response along $E[t]$ (length), training / held-out	0.66 / 0.49	0.75 / 0.75
response along the hidden direction c , training / held-out	0.005 / 0.006	0.24 / 0.24
follow by frequency, held-out: below 100 / 100-9,999 / 10,000+	0.34 / 0.79 / 0.83	0.62 / 0.90 / 0.90
edit test: self-report finite-change follow, range (mean)	0.73 to 0.90 (0.82)	0.72 to 0.91 (0.83)

Mean length of $E[t]$ by frequency (below 100 / 100-9,999 / 10,000+):

model	training tokens	held-out tokens
language model only	4.68 / 1.65 / 1.44	4.77 / 1.68 / 1.46
joint	1.53 / 1.44 / 1.33	3.95 / 1.75 / 1.41
control	4.11 / 1.70 / 1.41	4.13 / 1.75 / 1.43

Rescaling test on held-out vectors (score as in section 5.1):

held-out vectors	joint: as is $\rightarrow$ rescaled	control: as is $\rightarrow$ rescaled
rare, shrunk to the frequent length	-1.34 $\rightarrow$ 0.96	0.99 $\rightarrow$ 0.97
frequent, grown to the rare length	0.94 $\rightarrow$ -1.67	0.97 $\rightarrow$ -1.45

Observations.

1. Without the self-report's gradient on the embedding table, training and held-out tokens are read equally well (centred R^2 0.982 and 0.982; follow 0.871 and 0.868). The gap between them in the joint run is gone.
2. Rare tokens' vectors are long in the control whether asked about or not (4.11 and 4.13), as in the language-model-only model. So in the joint run, the self-report's gradient is what kept the asked-about rare vectors short.
3. The control reads the long rare vectors well (0.99), because long rare vectors are among its training tokens. It still fails on frequent vectors grown to the rare length (-1.45): what matters is whether a vector lies in a region the reader was trained on, not length as such. The rare vectors all point in nearly the same direction (mean pairwise cosine 0.998), so "long" means long in that one direction.
4. The control is far less blind along the hidden direction c (0.24 against 0.005) and follows length changes better (0.75).
5. The language model pays more in the control: 0.144 nats per token against 0.102.

Interpretation. In the joint runs, the self-report found a cheaper solution than reading: it changed the embedding vectors it was asked about so that its reader works on them, at less cost to the language model. That solution does not carry over to the tokens it was not asked about, whose vectors kept the shape the language model gave them. When the self-report cannot change the embedding table, it has to learn to read the vectors as the language model makes them, and then it reads tokens it was never asked about just as well. This is one seed of the control; the size of the effect (0.68 against 0.98) is much larger than the difference between the two joint seeds (0.68 and 0.50 are both far below 0.98).